

Bari-Joris BOICOS
Baptiste LAMBERT
Jonathan NGO
Tom CONTI
ING3 IA Groupe A

Rapport de projet

Agent conversationnel

Conception d'un agent conversationnel spécialisé dans le langage SQL

UE1 : Bases et outils de l'IA – NLP

Cours : NLP

Intervenant: Ian HANG

Date de rendu : 30 septembre

Année 2026-2027

1. Présentation du projet

L'objectif de ce projet est de concevoir un système d'intelligence artificielle capable de traduire des requêtes formulées en langage naturel (français) en requêtes de base de données structurées (SQL). Plutôt que de développer un algorithme de zéro, nous avons opté pour le ré-entraînement (Fine-Tuning) d'un grand modèle de langage (LLM) afin de le spécialiser dans cette tâche de traduction technique stricte.

2. Répartition des tâches

- Bari : Développer une classe pour observer le modèle, de la tokenisation à la prédiction

Baptiste : Construction d'une baseline conversationnelle et préparation des données

Jonathan : Génération de l'adaptateur LoRA et gestion de l'agent SQL

Tom : Création de l'interface Gradio et intégration de l'agent à l'interface.

3. Travail

3.1 Observation du modèle

Dans cette première partie, l'objectif est d'observer un LLM et de lier chaque étape de son processus à des concepts vus en classe.

Pour cela, et pour le reste du projet, nous avons choisi d'utiliser *Qwen2.5:1.5B-Instruct*. Ce modèle a été choisi pour sa légèreté et sa polyvalence –facilement adaptable avec un LoRA– De plus, il est déjà prêt à l'emploi en tant que ChatBot (modèle Instruct)

Le rapport complet et détaillé de l'observation du modèle se trouve sous `src/model_observation/model_observation_rapport.ipynb`

Tout d'abord, il fallait observer comment le modèle tokenise une question, ce qui nous a menés à nous intéresser aux algorithmes de tokenisation classiques comme le Byte Pair Encoding. Avec ceci, nous avons pu observer les embeddings des tokens par le modèle Qwen – des tests similaires ont été effectués sur d'autres modèles, notamment Mistral:8B – Ensuite, il a fallu observer : les logits générés et les méthodes de sélection du prochain token.

L'objectif final était, à la lumière des cours suivis, d'identifier les différents concepts qui caractérisent un LLM comme l'attention causale et l'unembedding.

3.2 Baseline conversationnelle

Pour établir notre modèle de référence (baseline) avant l'étape de Fine-Tuning, nous avons développé un script d'inférence basé sur le modèle *Qwen2.5:1.5B-Instruct*. Afin d'optimiser

l'utilisation de la mémoire et la gestion matérielle, le modèle a été chargé avec une précision bfloat16 et une allocation automatique sur le GPU/CPU (*device_map="auto"*).

Pour contraindre le modèle dans son rôle d'expert base de données, nous avons expérimenté plusieurs instructions système (system prompts). Nous avons finalement retenu une instruction stricte et détaillée imposant quatre règles fondamentales : répondre exclusivement en français, formater le code SQL dans des blocs dédiés, faire preuve de concision, et surtout refuser poliment toute requête hors-sujet (concept de "cas négatif").

La gestion de la mémoire à court terme de l'agent a été implémentée via une file d'historique limitée aux 8 derniers messages (soit les 4 derniers échanges complets). Cela permet de conserver le contexte de la conversation tout en évitant de saturer la fenêtre de contexte du modèle et la RAM. La génération des réponses est effectuée sous le contexte *torch.no_grad()* pour économiser la VRAM, avec des paramètres de décodage spécifiquement ajustés pour la génération de code : une température très basse (0.2) favorisant des réponses déterministes et précises, un top_p de 0.9, et une limite de longueur de 200 tokens.

3.3 Préparation des données

Afin d'entraîner notre agent avec l'adaptateur LoRA, nous avons constitué un jeu de données brut structuré sous forme de dialogues. Chaque exemple respecte un format conversationnel standardisé comprenant les rôles system, user, et assistant. Ce corpus intègre une grande diversité de scénarios : des requêtes SQL allant de simples sélections à des jointures complexes, des instructions de mise à jour ou de création de tables, mais aussi des exemples "hors périmètre" (recettes de cuisine, météo, code Python, traductions). Cette diversité est cruciale pour que le modèle apprenne non seulement la syntaxe SQL, mais aussi à appliquer la règle de refus définie dans le prompt système.

Pour assurer un apprentissage et une évaluation rigoureuse, nous avons utilisé la fonction *train_test_split* de la bibliothèque *scikit-learn* afin de diviser ce corpus en trois sous-ensembles distincts. La graine aléatoire a été fixée (*random_state=42*) pour garantir la reproductibilité de la division. La répartition appliquée est la suivante :

- **80%** des données dédiées à l'entraînement (Train).
- **10%** dédiées à la validation (Validation).
- **10%** mises de côté pour l'évaluation finale (Test).

Enfin, le script automatise l'exportation de ces données. Il utilise le module *os* pour résoudre dynamiquement les chemins absolus, créant un sous-dossier dédié (datasets) dans lequel les trois partitions sont sauvegardées de manière sécurisée sous forme de fichiers .json, avec un encodage UTF-8 assurant la bonne conservation des accents français.

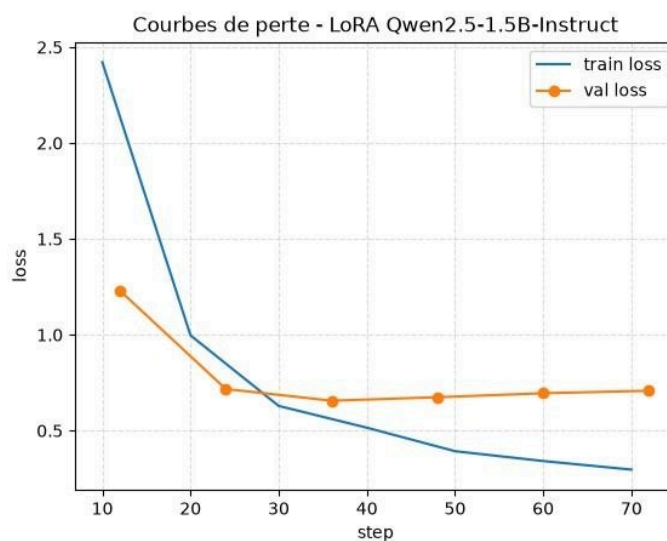
3.4 Adaptateur LoRA

Nous avons opté pour un adaptateur de type LoRA qui reste totalement gelé lors de la phase d'entraînement où seuls les paramètres du nouvel adaptateur sont entraînés.

Pour optimiser l'utilisation de la mémoire vidéo sans dégrader les performances avec une quantification extrême (QLoRA en 4-bit), nous avons chargé les poids du modèle en précision bfloat16.

L'adaptateur a été configuré avec un rang LoRA $r = 16$, ciblant une grande variété de modules (les projections de l'attention et le bloc MLP) afin de maximiser la capacité du réseau à assimiler la logique rigoureuse du langage SQL. Le processus d'apprentissage s'est étalé sur 6 époques et en bridant la longueur maximale des séquences à 512 tokens, ce qui est suffisant pour englober nos requêtes tout en préservant la RAM.

Afin de stabiliser mathématiquement le modèle tout en respectant nos limites matérielles, nous avons défini une taille de lot (batch size) de 4 par appareil, couplée à 4 étapes d'accumulation de gradient, permettant de simuler efficacement une taille de lot globale de 16. Enfin, le taux d'apprentissage a été fixé à $2e-4$ régulée par une courte phase d'échauffement initiale et une décroissance douce en cosinus.



4. Interface

Dans le cadre de ce projet, nous avons conçu une interface utilisateur afin de rendre intuitives les interactions avec notre modèle adapté. Pour cela nous avons utilisé la bibliothèque Gradio, cette dernière permettant de générer une interface simple rapidement. Après avoir structuré la mise en page de l'interface, que ce soit le titre, la description du modèle ou encore la place de la chatbox, nous avons ajouté la possibilité d'avoir des onglets différents pour pouvoir travailler sur plusieurs modèles à la fois. Puis nous avons configuré le composant de chatbox en évaluant les paramètres qui nous seraient utiles, notamment les paramètres permettant une sauvegarde de l'historique différentes en fonction du chat utilisé.

Le principal défi a résidé dans la gestion du multi-chat, la séparation initiale entre la

génération et l'historique au sein de la classe du modèle entrainé en conflit avec le fonctionnement interne de Gradio, ce qui a nécessité de modifier directement certaines fonctions de la classe SQLAgent.

5. Tests

Nous avons testé notre agent sur 20 exemples différents, voici un tableau résumant ces tests. Les requêtes précises sont dans

Requête testée	Pertinence et correction de la réponse	Respect du rôle et du format attendu	historique	Hors Périmètre	Infos inventées ?
Requête 1 <i>(Densité de population)</i>	Mauvaise. La logique mathématique que est inversée : la densité est Population / Surface, et non l'inverse.	Moyen. Donne la requête directement mais n'explique pas les étapes à un "amateur".	Non	Non	Oui
Requête 2 <i>(CA vendeurs mois dernier)</i>	Très mauvaise. Le DATE_SUB est placé de manière absurde dans le GROUP BY au lieu d'une clause WHERE	Moyen. Fournit une requête brute sans expliquer la logique des jointures à un débutant.	Non	Non	Oui (<i>Alias "latour" incompréhensible, table "factures" supposée</i>)

	pour filtrer les dates.				
Requête 3 <i>(Meilleurs produits / jeunes)</i>	Médiocre. A traduit subjective ment "jeune" par < 30 et "meilleur" par le plus cher (ORDER BY prix DESC) sans chercher à comprendre le besoin.	Mauvais. Un bon assistant aurait dû poser des questions de clarification ("Comment définissez-vous jeune ?") au lieu de deviner.	Non	Non	Oui <i>(Critères fixés arbitrairement)</i>
Requête 4 <i>(Lien avec la table PAYS)</i>	Très mauvaise. Parle de la fonction OVER() sans l'utiliser. Fait une jointure absurde sur des codes postaux. Ne calcule pas "par millier" (juste par habitant).	Mauvais. L'explication textuelle contredit la requête SQL générée.	Non (A oublié que la population était déjà dans PAYS depuis la R1)	Non	Oui (A inventé une table POPULATION_PAYS)

Requête 5 <i>(PC qui rame / Chrome)</i>	Bonne. Ne tombe pas dans le piège et refuse de générer une requête ou de donner des conseils IT.	Bon. Reste dans son personnage d'expert MySQL, bien que le ton soit un peu sec pour un assistant.	Non	Oui	Non
Requête 6 <i>(requête simple)</i>	Bonne. Donne une requête SQL simple qui effectue efficacement la demande.	Bon. Reste dans son personnage d'expert MySQL. Ne fais pas de commentaire particulier	Non	Non	Oui
Requête 7 <i>(ajout d'éléments dans une précédente requête)</i>	Bonne. L'assistant a bien intégré la demande à la requête précédente	Bon. Précise la modification principale apportée	Oui	Non	Oui
Requête 8 <i>(tarte aux pommes)</i>	Bonne. L'assistant précise sa spécialisation	Bon. L'assistant n'a pas donné d'information	Non	Non	Non

Requête 9 <i>(edtryftugyi hu)</i>	Bonne. L'assistant ne tente pas de répondre alors qu'il ne s'agit pas d'SQL	Bon. L'assistant n'essaie pas d'inventer une signification à la demande	Non	Non	Non
Requête 10 <i>(test de JOIN)</i>	Bonne. L'assistant fais bien le JOIN entre les deux table et imagine les tables qui ne lui ont pas été fournis	Bon.	Non	Oui	Oui
Requête 11 <i>(bonjour)</i>	Bonne. L'assistant répond dans son rôle d'assistant SQL	Moyen. L'assistant ne parle pas d'SQL mais ne s'étend pas sur le sujet	Non	Non	Oui, (L'assistant s'imagine s'appeler Marlon)
Requête 12 <i>(injection SQL simple)</i>	Moyenne. L'assistant a répondu une requête alors qu'il fallait refuser l'échange	Bon. L'assistant n'a pas exécuté l'injection.	Non	Non	Non

Requête 13 <i>(référence à un ancien prompt)</i>	Mauvaise. L'assistant n'a pas réussi à retrouver une réponse précédente .	Mauvais. La réponse ne présentait pas de signe "d'assistant SQL"	Non	Oui	Non
Requête 14 <i>(date négative)</i>	Mauvaise. L'assistant a remplacé la valeur négative par une autre date alors que ce n'était pas demandé	Moyen. L'assistant n'a pas pointé le problème dans la demande du client mais est resté dans son domaine de compétence.	Non	Oui	Oui
Requête 15 <i>(renommer et group by)</i>	Bonne. L'assistant a été capable de répondre à la demande	Bon. L'assistant a mis en évidence certaines fonctions importante de la requête	Non	Non	Oui
Requête 16	Bonne. L'assistant a bien répondu à la demande	Bon. L'explication est concise et la requête est bien	Non	Non	Non

<i>(update)</i>		isolée dans un bloc de code Markdown SQL.			
Requête 17 <i>(order by avec historique)</i>	Mauvaise. L'assistant écrit une faute technique	Moyen. Le format est respecté , mais l'assistant applique aveuglément l'instruction de tri sans avertir l'utilisateur que cela n'a pas d'utilité directe sur le rendu visuel de la modification.	Oui	Non	Oui
Requête 18 <i>(refus requête python)</i>	Bonne. L'assistant applique correctement sa consigne de refus face à une demande de code dans un autre langage (Python).	Bon. Il reste dans son rôle d'expert strict en MySQL et réoriente poliment l'utilisateur vers une syntaxe purement SQL.	Non	Oui	Non
Requête 19 <i>(calculs et fonctions avancées)</i>	Mauvaise. L'assistant donne une requête bien plus complexe que prévu.	Moyen. L'assistant livre une requête surdimensionnée et difficilement lisible.	Non	Non	Oui
Requête 20	Mauvaise. L'énoncé demandait de compter des	Moyen. L'assistant ne répond pas précisément	Non	Non	Oui

(recherche d'un client)	individus, mais le modèle génère une requête de sélection globale avec des filtres. Et il se trompe de nom/prénom .	à la problématique de comptage formulée dans la consigne.			
-------------------------	---	---	--	--	--